

Lecture 9 — 12th September

Lecturer: Arjun Bhagoji

Scribe: Anurag Deshpande

9.1 Attack Strategy: Bilevel Optimization

Bilevel optimization [4] is a powerful and general framework for formulating data poisoning attacks. It is applicable across various scenarios, including indiscriminate, targeted, clean-label, and backdoor attacks. The primary requirement for this strategy is that the attacker has access to the training data and the model architecture.

9.1.1 General Formulation

The attack is structured as a nested optimization problem. The outer loop optimizes the poisoned data to achieve the attacker's goal, while the inner loop simulates the training of the model on this data. We define the following notation:

- D_{train} : The full original training dataset.
- D_p : The subset of training data that can be poisoned by the attacker.
- D_c : The clean, unmodifiable portion of the training data.
- D_v : A clean validation set, drawn i.i.d. from the true data distribution, used to define the attacker's objective.
- $M(\cdot)$: A manipulation function that defines the set of allowed perturbations.
- $\theta \in \mathbb{R}^d$: The model parameters. We denote parameters trained on poisoned data as $\tilde{\theta}_p$.

The general bilevel optimization problem for a targeted attack is formulated as:

$$\begin{aligned} & \underset{\tilde{D}_p \in M(D_p)}{\text{maximize}} && \hat{L}(D_v, \tilde{\theta}_p) \end{aligned} \tag{1}$$

$$\text{subject to} \quad \tilde{\theta}_p \in \underset{\theta}{\text{argmin}} \quad \hat{L}(D_c \cup \tilde{D}_p, \theta) \tag{2}$$

Here, we maximize the loss on the validation set D_v to cause misclassification of target examples.

9.1.2 Indiscriminate Poisoning

The goal is to find a poisoned dataset $\tilde{D}_p$ that maximizes the loss on a clean dataset when the model is trained on the poisoned data.

$$\begin{aligned} & \underset{\tilde{D}_p \in M(D_p)}{\text{maximize}} && \mathbb{E}_{\tilde{z} \sim P^*} [l(\tilde{z}, \tilde{h})] \\ & \text{subject to} && \tilde{h} \in \underset{h}{\text{argmin}} L(D_c \cup \tilde{D}_p, h) \end{aligned} \tag{3}$$

9.1.3 Targeted Poisoning

In targeted poisoning, the attacker aims to cause misclassification for a specific target instance.

$$\begin{aligned} & \min_{\tilde{D}_p \in M(D_p)} \hat{L}(D_v, \tilde{h}) \\ \text{subject to } & \tilde{h} \in \arg \min_h \hat{L}(D_c \cup \tilde{D}_p, h) \end{aligned} \quad (4)$$

9.1.4 Backdoor Attacks

Backdoor attacks involve inserting a trigger into the training data. The model learns to associate the trigger with a specific target label.

$$\begin{aligned} & \min_{\tilde{D}_p \in M(D_p)} \hat{L}(D_c \cup \tilde{D}_p, h) \\ \text{subject to } & \tilde{h} \in \arg \min_h \hat{L}(D_c \cup \tilde{D}_p, h) \end{aligned} \quad (5)$$

It is important to note that the triggers used during training and testing are usually the same.

9.1.5 Solving the Bilevel Optimization Problem

A key question is how to solve the bilevel optimization problem efficiently [3]. Let's simplify the notation. Consider a single poison point $(\vec{x}^p, y^p)$. We want to solve:

$$\begin{aligned} & \arg \max_{\tilde{D}_p \in M(D_p)} \frac{1}{n_v} \sum_{i=1}^{n_v} l(z_i, \vec{\theta}_p) \\ \text{subject to } & \theta_p \in \arg \min_{\theta} \frac{1}{n_c + n_p} \left(\sum_{i=1}^{n_c} l(z_i, \vec{\theta}_p) + \sum_{i=1}^{n_p} l(z_i^*, y_i^*, \vec{\theta}_p) \right) \end{aligned} \quad (6)$$

- **Single Poison Point:** We assume the adversary controls only a single poison point $(\mathbf{x}_p, y_p)$ and fixes the poison label y_p throughout the attack. The optimization is performed over the features $\mathbf{x}_p$.
- **Loss Function Properties:** The loss function $l(z, \theta)$ is assumed to be twice differentiable and convex with respect to the parameters θ , ensuring the Hessian is positive definite ($\nabla_{\theta}^2 l(z, \theta) \succ 0$).

9.1.6 Gradient Ascent Attack

With these assumptions, the outer maximization problem for the single poison point can be reformulated, and simply stated, and becomes:

$$\text{maximize}_{\mathbf{x}_p \in M(\cdot)} \hat{L}(D_v, \vec{\theta}_p(\mathbf{x}_p)) \quad (7)$$

We can solve this using **Projected Gradient Ascent**. At each step t , the poison features $\mathbf{x}_p$ are updated as follows:

$$\vec{x}_t^p = \vec{x}_{t-1}^p + \eta \nabla_{\vec{x}_{t-1}^p} \hat{L}(D_v, \vec{\theta}_p(\vec{x}_{t-1}^p)) \quad (8)$$

where η is the learning rate, and the updated point is projected back into the allowed manipulation set (e.g., to maintain image validity). The main challenge is computing the gradient $\nabla_{\mathbf{x}_p} \hat{L}(\cdot)$.

9.1.7 Chain Rule and Implicit Differentiation

Using the chain rule, the gradient of the validation loss w.r.t. the poison data $\mathbf{x}_p$ is:

$$\nabla_{\mathbf{x}_p} \hat{L}(D_v, \tilde{\theta}_p(\mathbf{x}_p)) = \underbrace{\left(\frac{\partial \tilde{\theta}_p(\mathbf{x}_p)}{\partial \mathbf{x}_p} \right)^T}_{\text{Term 1: Jacobian}} \underbrace{\nabla_{\tilde{\theta}_p} \hat{L}(D_v, \tilde{\theta}_p(\mathbf{x}_p))}_{\text{Term 2: Validation Gradient}} \quad (9)$$

Term 2 is easy to compute via backpropagation. Term 1, the Jacobian, describes how the learned parameters change with respect to the poison data. To compute it, we use the **stationarity condition** from the inner optimization problem (Eq. (2)), which states that the gradient of the training loss at the optimal parameters $\tilde{\theta}_p$ is zero:

$$\nabla_{\tilde{\theta}_p} \hat{L}(D_c \cup \{(\mathbf{x}_p, y_p)\}, \tilde{\theta}_p(\mathbf{x}_p)) = 0 \quad (10)$$

By the implicit function theorem, we can differentiate this entire equation with respect to $\mathbf{x}_p$:

$$\frac{d}{d\mathbf{x}_p} \left[\nabla_{\tilde{\theta}_p} \hat{L}(D_c \cup \{z_p\}, \tilde{\theta}_p) \right] = 0 \quad (11)$$

Applying the total derivative yields:

$$\nabla_{\mathbf{x}_p} \nabla_{\tilde{\theta}_p} \hat{L}(D_{train}, \tilde{\theta}_p) + \left(\frac{\partial \tilde{\theta}_p}{\partial \mathbf{x}_p} \right)^T \nabla_{\tilde{\theta}_p}^2 \hat{L}(D_{train}, \tilde{\theta}_p) = 0 \quad (12)$$

Let $H_{\tilde{\theta}} = \nabla_{\tilde{\theta}_p}^2 \hat{L}(D_{train}, \tilde{\theta}_p)$ be the Hessian of the training loss. We can now solve for the Jacobian term:

$$\left(\frac{\partial \tilde{\theta}_p}{\partial \mathbf{x}_p} \right)^T = - \left[\nabla_{\mathbf{x}_p} \nabla_{\tilde{\theta}_p} \hat{L}(D_{train}, \tilde{\theta}_p) \right] H_{\tilde{\theta}}^{-1} \quad (13)$$

Since $\mathbf{x}_p$ only appears in one term of the loss, this simplifies to $\nabla_{\mathbf{x}_p} \nabla_{\tilde{\theta}_p} l(z_p, \tilde{\theta}_p)$. Substituting Eq. (13) into the chain rule (Eq. (9)) gives the final expression for the attack gradient. Further steps are possible to speed up computation [1]

9.2 Attack Strategy: Feature Collision

This is a targeted attack applicable in clean and backdoor scenarios, requiring both data and model access, where the model can only be fine-tuned. The attacker aims to create a poison instance that has a similar feature representation to a target instance.

The poison point $\vec{x}^p$ is found by solving the following optimization problem:

$$\vec{x}^p = \arg \min_{\vec{x} \in \mathbb{R}^k} [\|\phi(\vec{x}) - \phi(\vec{x}^t)\|_2^2 + \lambda \|\vec{x} - \vec{x}^c\|_2^2]$$

where ϕ is a known feature extractor, $\vec{x}^t$ is the target instance, and $\vec{x}^c$ is a clean base instance.

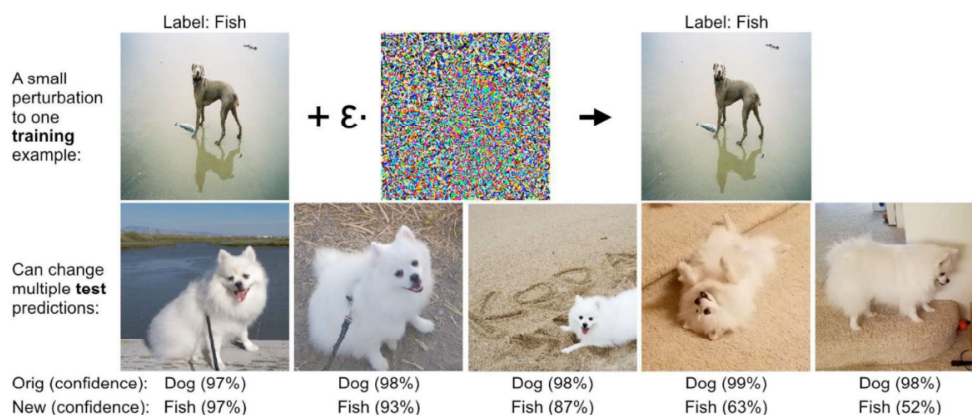


Figure 9.1. A small perturbation to one training example can change multiple test predictions. The model used is an Inception architecture where only the last layer is retrained at each iteration.

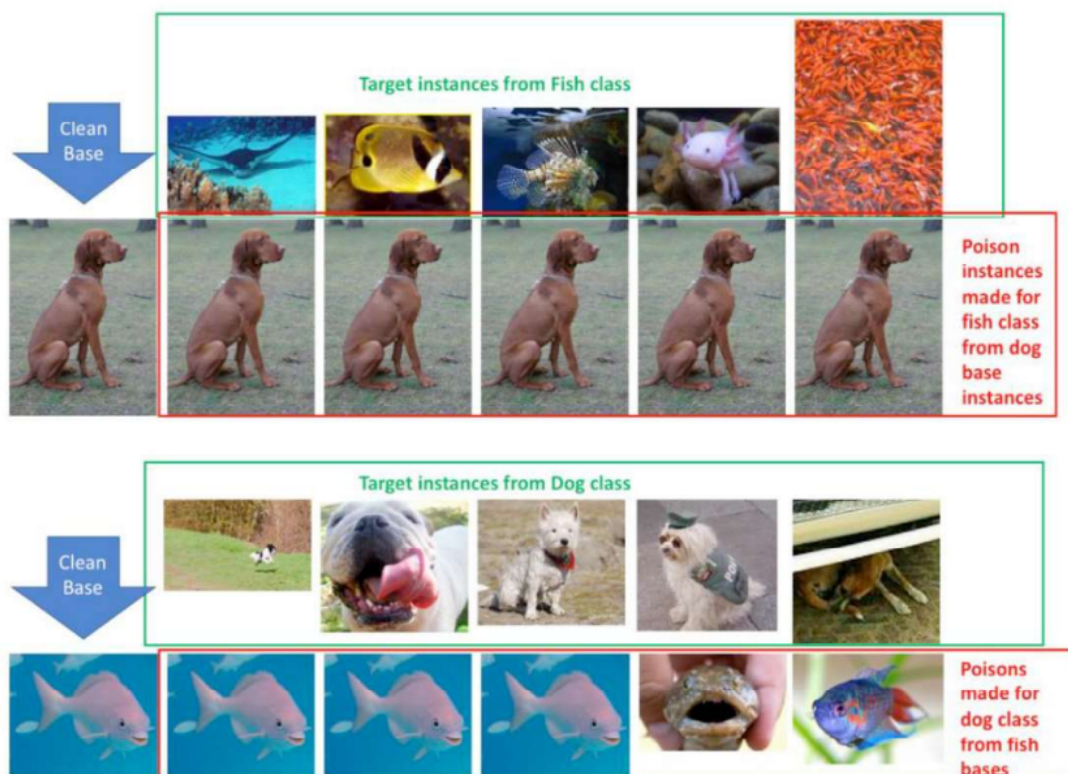


Figure 9.2. Results of feature collision attacks. (a) shows sample target and poison instances. (b) shows that poisoned models misclassify the target with high confidence.

9.3 Attack Strategy: Gradient Alignment

This is a targeted attack for clean and backdoor scenarios that requires both data and model access. The core idea is to craft a poisoned image whose gradient aligns with the gradient of the loss for the target image. [2]

Thus, the optimization problem is formulated as:

$$\min_{\tilde{D}_p \in M(D_p)} 1 - \frac{\langle \nabla_{\theta} l(h_{\theta}(\vec{x}^t), y^p), \nabla_{\theta} \hat{L}(\tilde{D}_p, \vec{\theta}) \rangle}{\|\nabla_{\theta} l(h_{\theta}(\vec{x}^t), y^p)\| \cdot \|\nabla_{\theta} \hat{L}(\tilde{D}_p, \vec{\theta})\|} \quad (14)$$

where h_{θ} is a pre-trained clean model.

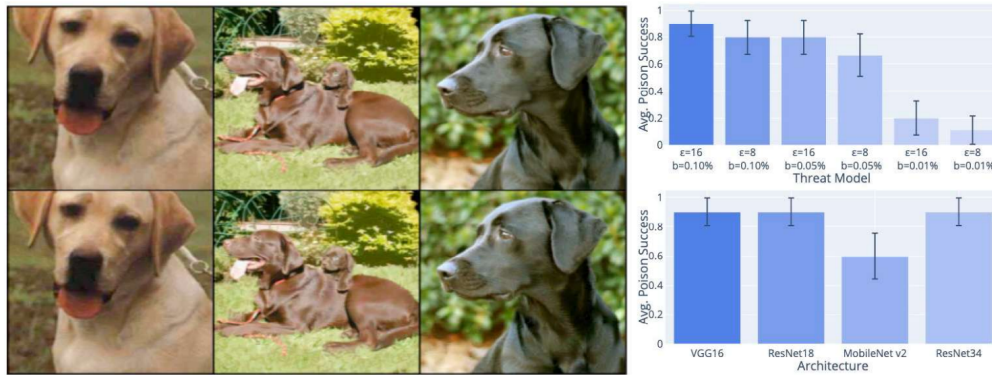


Figure 9.3. Poisoning ImageNet using Gradient Alignment. The left panel shows clean images and their poisoned counterparts. The right panels show the success rate for different poison budgets and model architectures.

9.4 Empirical considerations

- **Poison fraction sensitivity:** Typical curves show test accuracy vs poison fraction (0% to 20%), with steep drops in some regimes.
- **Label flip vs clean-label:** Label-flip often simpler but easier to detect; clean-label often requires optimization that preserves perceptual similarity.
- **Defense notes:** Common defenses include data sanitization, robust training, and influence-based detection.

Bibliography

- [1] Pang Wei Koh and Percy Liang. Understanding Black-box Predictions via Influence Functions. arXiv preprint arXiv:1703.04730, 2017.
- [2] Duc-Tuan Truong, Tianchi Liu, Junjie Li, Ruijie Tao, Kong Aik Lee, and Eng Siong Chng. Addressing gradient misalignment in data-augmented training for robust speech deepfake detection, 2025.
- [3] Mao Ye, Bo Liu, Stephen Wright, Peter Stone, and Qiang Liu. Bome! bilevel optimization made easy: A simple first-order approach, 2022.
- [4] Yekun Zhang, Peyman Khanduri, Ioannis Tsaknakis, Yuan Yao, Mingyi Hong, and Shuzhong Liu. An Introduction to Bi-level Optimization: Foundations and Applications in Signal Processing and Machine Learning. arXiv preprint arXiv:2308.00788, 2023.